

Proyecto Base: Mercado Fresco — Almacén de Barrio

Materia: Paradigmas y Lenguajes de Programación III

Opción Seleccionada: Opción 3 — Buscador y Filtro de Catálogo en Tiempo Real

Alumno: Santiago Romero Baylac

1. Introducción y Alcance

En el marco de la integración práctica sobre el proyecto del TP1 ("Mercado Fresco"), se implementó una refactorización integral sobre la vista de catálogo en tarjetas ([listado_box.html](#)).

El propósito fue sustituir la renderización estática de contenidos mediante la incorporación de llamadas HTTP asíncronas con la API nativa `fetch`, la manipulación programática del DOM con JavaScript estándar (ES6+) y la vinculación desacoplada de eventos de usuario, sustentando técnicamente la arquitectura de navegación mediante la metodología de diseño hipermedial **SOHDM**.

2. Requerimientos Técnicos Implementados

2.1 Desacoplamiento de Eventos

- Se erradicó por completo el uso de manejadores de eventos incrustados en etiquetas HTML (tales como `onclick` u `onsubmit`).
-
- La interceptación de las acciones del usuario se centralizó en el archivo externo `app.js` mediante el método estándar `addEventListener()`.
-
- Para la búsqueda interactiva, se capturó el evento `input` sobre el campo de texto, permitiendo una respuesta reactiva inmediata ante cada pulsación del usuario sin requerir el envío manual de formularios ni recargas de página.
-

2.2 Asincronía y Manejo de Datos (Fetch API)

- Los datos de los artículos, anteriormente definidos en un array estático dentro de un script de cliente, fueron externalizados al archivo estandarizado `productos.json`.
-
- La obtención de los recursos se estructuró a través de una función asíncrona empleando la sintaxis `async / await` en combinación con la función nativa `fetch("./productos.json")`.
-
- Se implementaron bloques de control de excepciones `try...catch` junto a la verificación de la cabecera de respuesta (`respuesta.ok`) para garantizar una degradación grácil de la interfaz frente a eventuales fallos de red o errores de ruta (404/500).
-

2.3 Procesamiento en Memoria y Mutación del DOM

- Una vez resuelta la promesa del archivo JSON, los datos se almacenan en un array local (`catalogo`).
-
- El filtrado en tiempo real opera evaluando las coincidencias por coincidencia parcial de texto mediante el método funcional `catalogo.filter()`, analizando simultáneamente los campos `name` y `category`.
-
- El contenedor principal (`.box-grid`) se actualiza dinámicamente vaciando su árbol interno e inyectando nodos `<article>` construidos programáticamente, preservando los atributos de datos (`data-*`), imágenes emoji, precios formateados y badges visuales de descuento y stock.
-

3. Detalle de Archivos Modificados y Creados

3.1 `productos.json` (Nuevo)

Archivo de intercambio de datos que opera como repositorio central del catálogo. Almacena las entidades con sus propiedades tipadas: `id`, `name`, `category`, `categorySlug`, `price`, `stockQty`, `stockLevel`, `emoji`, `description` y flags de oferta (`discount`).

3.2 `listado_box.html` (Modificado)

- **Inserción de Interfaz de Búsqueda:** Se introdujo un campo interactivo `<input type="text" id="input-buscador">` en la cabecera del catálogo.
-
- **Limpieza de Contenido Estático:** Se eliminaron las más de 170 líneas correspondientes a las tarjetas de productos hardcodeadas, delegando la totalidad del dibujo al script.
-
- **Eliminación de Scripts Embebidos:** Se removió el bloque `<script>` que contenía lógica procedural inline al pie del documento, sustituyéndolo exclusivamente por la referencia desacoplada `<script src="app.js"></script>`.
-

3.3 `app.js` (Nuevo)

Controlador principal de la vista que orquesta el ciclo de vida del módulo:

1. Escucha la inicialización del DOM mediante `DOMContentLoaded`.
- 2.
3. Ejecuta `cargarCatalogo()` para invocar asincrónicamente a `fetch("./productos.json")`.
- 4.
5. Procesa parámetros de consulta de la URL (`URLSearchParams`) para mantener la compatibilidad con filtros directos de categoría existentes (`?cat=` o `?filter=ofertas`).
- 6.
7. Vincula el evento `input` al buscador para ejecutar el filtrado sobre el array y disparar `renderizarTarjetas()` de manera reactiva.
- 8.

4. Fundamentación Metodológica — Modelo SOHDM

(Scenario-based Object-Oriented Hypermedia Design Methodology)

4.1 Escenario de Exploración y Búsqueda

En la metodología SOHDM, el proceso de especificación se rige por escenarios de usuario contextualizados (*User Scenarios*). Para el módulo de catálogo de "Mercado Fresco", se formalizó el **Escenario de Exploración y Búsqueda Rápida**. El objetivo operacional de este escenario es permitir al cliente recorrer y reducir el inventario disponible de víveres en función de sus intenciones de consumo inmediatas, eliminando la fricción y el costo cognitivo de navegación que implican las peticiones síncronas convencionales.

4.2 Estructura y Dinámica del Mapa Navegacional

- **Nodo Índice Base:** La vista `listado_box.html` representa el contexto navegacional raíz (nodo de clase estructural), el cual presenta la totalidad de las instancias de la clase de dominio `Producto`.
-
- **Poda Dinámica del Subgrafo Navegacional:** En los diseños hipermediales clásicos, una búsqueda implica la transición hacia un nodo nuevo o página de resultados (un salto en el mapa navegacional). SOHDM resuelve este comportamiento como una **transformación de estado intra-contextual**: la interacción del usuario sobre el control de búsqueda genera una restricción dinámica sobre el contexto activo. El filtro en memoria (`.filter()`) poda de forma selectiva las unidades no relevantes del árbol DOM sin abandonar el nodo, minimizando la carga sobre la red y conservando la continuidad espacial del usuario.
-

4.3 Separación Estricta entre Capas

La solución respeta los postulados arquitectónicos de SOHDM al desacoplar el modelo de información del modelo navegacional y de presentación:

- **Modelo Conceptual (Datos):** Residenciado en `productos.json`, define el esquema puro de atributos y tipos de las entidades de dominio sin dependencias de diseño o maquetación.
-
- **Modelo Navegacional y de Interfaz:** Controlado mediante `app.js` y `styles.css`, donde la lógica de renderizado proyecta las tarjetas (`product-card`) y establece los puntos de interacción de forma desacoplada, asegurando que futuras modificaciones en las vistas no afecten la consistencia de los datos subyacentes.